Scan2BIM Low Level Design

This page documents the low level design for the Scan2BIM processing workflow that starts after a user uploads an `.e57` point cloud file. It describes the internal service interactions, queue flow, workflow orchestration, status tracking, retry behavior, and final IFC output handling.

> **Architecture style:** Event-driven, queue-based processing with workflow orchestration through Temporal, asynchronous service-to-service communication through Azure Service Bus, and centralized job tracking in MongoDB.

Purpose and Scope

The objective of this design is to define how Scan2BIM services collaborate to process uploaded point cloud data into a final IFC file. The design focuses on service boundaries, message movement, state transitions, persistence, and operational behavior.

In scope

- Upload event handling
- Temporal workflow initiation
- Azure Service Bus queue-based processing
- Segmentation execution
- Post-processing execution

Out of scope

- Detailed ML model internals
- ESDT Portal UI design
- Infrastructure sizing
- Detailed AKS deployment YAML
- Database indexing strategy

- IFC creation execution
- IFC upload execution
- MongoDB job status tracking
- Final IFC file storage
- Production monitoring dashboard design

Architecture Overview

The Scan2BIM workflow begins when the Data Repository emits an upload completion event for an `.e57` file. The Orchestrator Microservice validates the request, creates a workflow context, starts a Temporal workflow, stores the initial job record in MongoDB, and publishes a message to the Segmentation Queue.

Each downstream microservice consumes a message from its dedicated Azure Service Bus queue, performs its stage-specific processing, updates status in MongoDB, signals progress to Temporal, and publishes the next message for the next stage. The workflow ends when the IFC file is uploaded to final storage and the job is marked complete.

> **Key design principle:** Each stage is isolated and independently scalable. Services communicate through durable messages rather than direct synchronous calls, which improves resiliency, fault isolation, and throughput control.

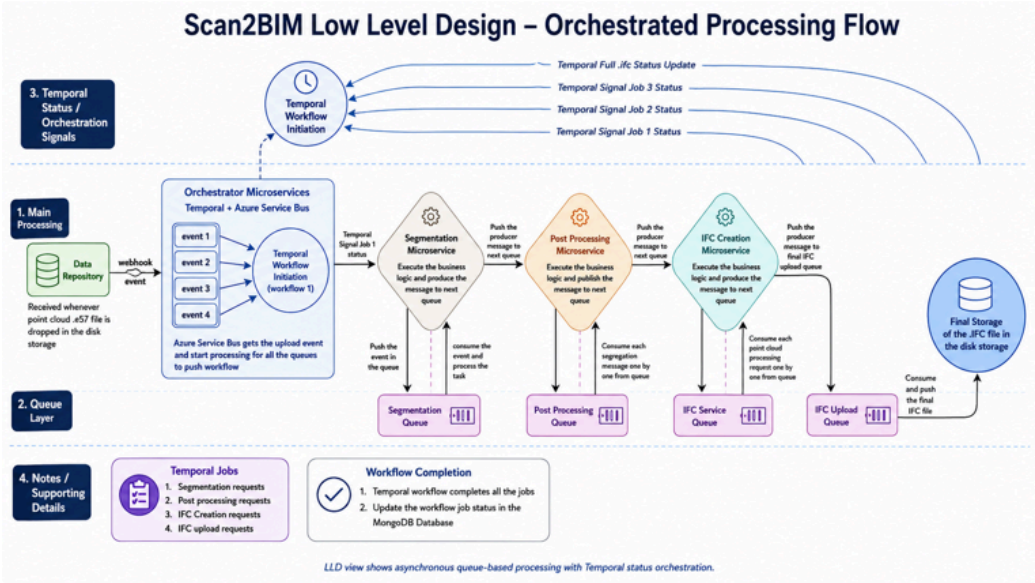
End-to-End Processing Flow

1. User uploads an `.e57` file to the Data Repository.
2. Data Repository sends a webhook event to the Orchestrator Microservice.
3. Orchestrator validates metadata such as project ID, site ID, file ID, and file path.
4. Orchestrator creates a unique `jobId` and `workflowId`.
5. Orchestrator starts the Temporal workflow.
6. Orchestrator writes the initial job record to MongoDB.
7. Orchestrator publishes a segmentation message to Azure Service Bus.
8. Segmentation Microservice consumes the job, reads the `.e57` file, generates segmentation output, updates state, and publishes the next message.
9. Post Processing Microservice consumes segmentation output, applies business logic, generates structured JSON output, updates state, and publishes the IFC creation message.
10. IFC Creation Microservice consumes processed output, generates the IFC file, updates state, and publishes the upload message.

11. IFC Upload Processing consumes the upload message and uploads the IFC file to final storage.
12. MongoDB and Temporal are updated throughout the lifecycle until the workflow is complete.

Low Level Flow Diagram

The following Mermaid flow can be pasted into a Confluence Mermaid macro.



Component Design

Data Repository

The Data Repository is the source location for the uploaded point cloud file and associated metadata. It holds the uploaded `.e57` file reference and publishes the upload completion event that starts the workflow.

- **Stores:** file reference, project metadata, site metadata, job-relevant identifiers
- **Provides:** source file path for segmentation
- **Triggers:** webhook event to the Orchestrator

Orchestrator Microservice

The Orchestrator is the control entry point for the workflow. It does not perform heavy processing. Its role is to validate the event, initialize workflow tracking, and trigger the first queued stage.

- Receives upload event

- Validates mandatory metadata
- Creates `jobId` and `workflowId`
- Starts the Temporal workflow
- Publishes message to Segmentation Queue
- Creates initial MongoDB status record

Temporal Workflow

Temporal manages the long-running workflow lifecycle. It tracks the progress of each stage, captures signals from services, manages retry logic at workflow level, and determines final completion or failure.

Area	Description
Workflow initiation	Starts processing after upload validation
Status tracking	Tracks each stage and overall workflow state
Signal handling	Receives job status updates from downstream services
Retry management	Applies configured retry policy for workflow activities
Failure handling	Marks the workflow failed if a stage permanently fails
Completion tracking	Marks workflow complete after final upload succeeds

Tracked job types: Segmentation Request, Post Processing Request, IFC Creation Request, and IFC Upload Request.

Azure Service Bus

Azure Service Bus provides decoupled asynchronous communication between services. Each stage has its own dedicated queue. A service consumes from one queue and publishes to the next queue only after successful execution of its current stage.

Queue Name	Producer	Consumer	Purpose

Segmentation Queue	Orchestrator	Segmentation Microservice	Starts segmentation processing for uploaded `.e57` input
Post Processing Queue	Segmentation Microservice	Post Processing Microservice	Starts structured output and business rule processing
IFC Service Queue	Post Processing Microservice	IFC Creation Microservice	Starts IFC generation
IFC Upload Queue	IFC Creation Microservice	Upload Processing	Triggers final IFC upload to storage

Segmentation Microservice

This service performs the first processing stage. It reads the uploaded point cloud, applies segmentation and classification logic, stores or references intermediate artifacts, updates workflow state, and triggers the next stage.

- Consumes segmentation message
- Reads uploaded `.e57` file
- Executes segmentation logic
- Produces segmentation output
- Publishes message to Post Processing Queue
- Updates MongoDB and signals Temporal

Status lifecycle: `SEGMENTATION_QUEUED`, `SEGMENTATION_IN_PROGRESS`, `SEGMENTATION_COMPLETED`, `SEGMENTATION_FAILED`

Post Processing Microservice

This service converts segmentation output into structured, business-usable data. It applies rules and transformations required for IFC generation.

- Consumes post-processing message
- Reads segmentation output
- Applies business rules
- Generates structured JSON outputs and component metadata
- Publishes message to IFC Service Queue

- Updates MongoDB and signals Temporal

Status lifecycle: `POST_PROCESSING_QUEUED`, `POST_PROCESSING_IN_PROGRESS`, `POST_PROCESSING_COMPLETED`, `POST_PROCESSING_FAILED`

IFC Creation Microservice

This service generates the BIM-compliant IFC file from processed JSON and business output. Based on current understanding, IFC generation may rely on DLL-based integration.

- Consumes IFC creation message
- Reads processed output
- Generates IFC file
- Publishes message to IFC Upload Queue
- Updates MongoDB and signals Temporal

Status lifecycle: `IFC_CREATION_QUEUED`, `IFC_CREATION_IN_PROGRESS`, `IFC_CREATION_COMPLETED`, `IFC_CREATION_FAILED`

IFC Upload Processing

This stage moves the generated IFC artifact to the final storage location and closes the workflow.

- Consumes upload message
- Locates generated IFC file
- Uploads IFC file to Blob or approved file storage
- Updates MongoDB final state
- Signals Temporal completion

Status lifecycle: `IFC_UPLOAD_QUEUED`, `IFC_UPLOAD_IN_PROGRESS`, `IFC_UPLOAD_COMPLETED`, `IFC_UPLOAD_FAILED`, `WORKFLOW_COMPLETED`

MongoDB Design

MongoDB acts as the operational source of truth for job progress and error visibility. It stores workflow metadata, current stage, per-stage status, retry count, and output location.

Collection: `scan2bim_jobs`

Field	Description
`jobId`	Unique job identifier

`workflowId`	Temporal workflow identifier
`projectId`	Project identifier
`siteId`	Site identifier
`fileId`	Uploaded file identifier
`fileName`	Uploaded `.e57` file name
`filePath`	Source input path
`currentStage`	Current stage being executed
`overallStatus`	Overall workflow status
`segmentationStatus`	Segmentation stage status
`postProcessingStatus`	Post-processing stage status
`ifcCreationStatus`	IFC creation stage status
`uploadStatus`	Final upload status
`outputFilePath`	Final IFC output location
`errorCode`	Error code for failed processing
`errorMessage`	Error details
`retryCount`	Number of retries performed
`createdAt`	Job creation timestamp
`updatedAt`	Last status update timestamp

Sample MongoDB document


```json
{
  "jobId": "JOB-12345",
  "workflowId": "SCAN2BIM-WF-12345",
  "projectId": "PRJ-001",
  "siteId": "SITE-001",
  "fileId": "FILE-001",
  "fileName": "site_scan.e57",
  "filePath": "/input/site_scan.e57",
  "currentStage": "IFC_UPLOAD",
  "overallStatus": "IN_PROGRESS",
  "segmentationStatus": "COMPLETED",
  "postProcessingStatus": "COMPLETED",
  "ifcCreationStatus": "COMPLETED",
  "uploadStatus": "IN_PROGRESS",
  "outputFilePath": null,
```

```
  "errorCode": null,
  "errorMessage": null,
  "retryCount": 0,
  "createdAt": "2026-06-12T10:00:00Z",
  "updatedAt": "2026-06-12T10:30:00Z"
}
```


Queue Message Contract

Each queue message must contain enough information for the next service to process the job independently without needing synchronous context lookups from upstream services.

Field	Mandatory	Description
`jobId`	Yes	Unique job identifier
`workflowId`	Yes	Temporal workflow identifier
`projectId`	Yes	Project identifier
`siteId`	Yes	Site identifier
`fileId`	Yes	File identifier
`sourceFilePath`	Yes	Input file location
`stage`	Yes	Current processing stage
`correlationId`	Yes	End-to-end trace identifier
`createdAt`	Yes	Message creation timestamp

Sample message payload


```
{
  "jobId": "JOB-12345",
  "workflowId": "SCAN2BIM-WF-12345",
  "projectId": "PRJ-001",
  "siteId": "SITE-001",
  "fileId": "FILE-001",
  "sourceFilePath": "/input/site_scan.e57",
  "inputArtifactPath": "/intermediat
```